

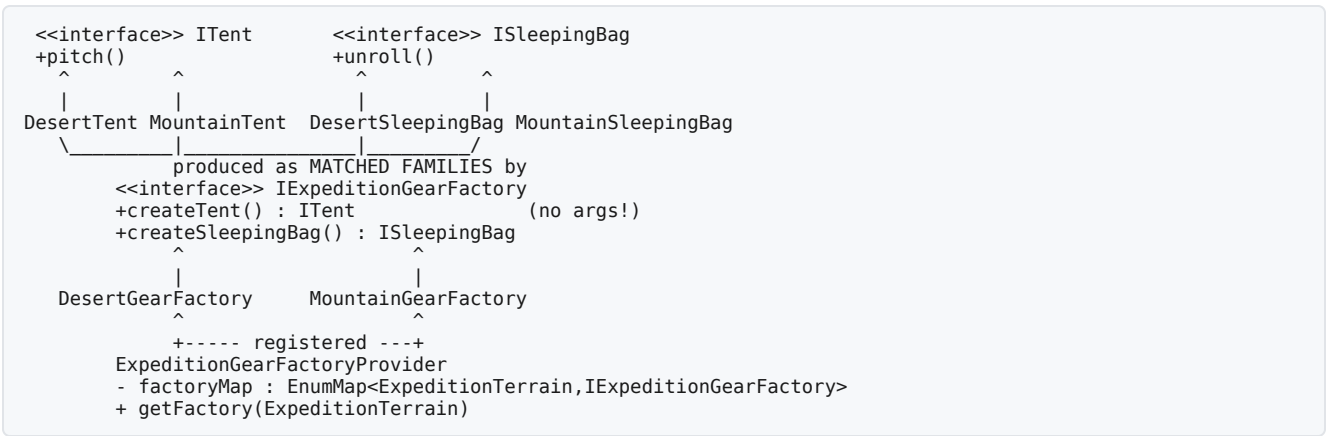
Abstract Factory Design Pattern

Intent

Provide an interface for creating **families of related objects** without specifying their concrete classes — and guarantee that everything a given factory produces belongs to the **same family**.

Here the families are **DESERT** and **MOUNTAIN** expedition gear. If you pick the mountain factory, every product it hands back (tent *and* sleeping bag) is mountain gear. You physically cannot end up with a desert tent next to a mountain sleeping bag. That guarantee — *consistency across a product family* — is the whole reason Abstract Factory exists and is what separates it from a plain Factory Method.

UML class diagram



The players

enums/ExpeditionTerrain	the family key (DESERT, MOUNTAIN)
product/ITent	abstract product A
product/ISleepingBag	abstract product B
product/concrete/DesertTent, MountainTent	concrete product A, one per family
product/concrete/DesertSleepingBag, MountainSleepingBag	concrete product B, one per family
factory/IExpeditionGearFactory	the ABSTRACT FACTORY – makes a tent AND a sleeping bag
factory/concrete/DesertGearFactory	concrete factory for the DESERT family
factory/concrete/MountainGearFactory	concrete factory for the MOUNTAIN family
factory/ExpeditionGearFactoryProvider	a registry that picks a factory by ExpeditionTerrain

Read the two dimensions carefully — this is the part people get wrong:

- **Product kind** — tent vs. sleeping bag (the *rows*).
- **Family** — desert vs. mountain (the *columns*).

An Abstract Factory has one concrete factory **per column**, and each produces one product **per row**. The grid:

	ITent	ISleepingBag
DesertGearFactory	DesertTent	DesertSleepingBag
MountainGearFactory	MountainTent	MountainSleepingBag

The code, line by line

ExpeditionTerrain — the family key

```
public enum ExpeditionTerrain {
    DESERT("DESERT"), MOUNTAIN("MOUNTAIN");
}
```

```

private final String name;
ExpeditionTerrain(final String name) { this.name = name; }

@JsonValue
public String getName() { return name; }
}

```

- A **single** enum names the families. It is the one axis of variation. Tents, sleeping bags, and factories are all keyed by this same enum, which keeps the model coherent.
- `private final String name` — final because an enum constant's label should never change. (A naive first draft might use `public String name`, which both leaks the field and shadows the built-in `Enum.name()` method — that mistake is avoided here from the start.)
- `@JsonValue getName()` — API-friendly JSON value; not needed by the pattern.

ITent / ISleepingBag — the abstract products

```

public interface ITent {
    ExpeditionTerrain getExpeditionTerrain();
    void pitch();
}
public interface ISleepingBag {
    ExpeditionTerrain getExpeditionTerrain();
    void unroll();
}

```

- Each abstract product exposes its behavior (`pitch()`, `unroll()`) plus `getExpeditionTerrain()` so it can **declare which family it belongs to**. That self-declaration is what lets the provider index things automatically, exactly like the Factory module.

Concrete products

```

@Component
public class DesertTent implements ITent {
    @Override public void pitch() { System.out.println("Pitching a desert tent!"); }
    @Override public ExpeditionTerrain getExpeditionTerrain() { return ExpeditionTerrain.DESERT; }
}

```

- `@Component` so Spring can inject them into the matching factory.
- `DesertTent / DesertSleepingBag` report `DESERT`; `MountainTent / MountainSleepingBag` report `MOUNTAIN`. Their family identity is baked in.

IExpeditionGearFactory — the abstract factory

```

public interface IExpeditionGearFactory {
    ExpeditionTerrain getExpeditionTerrain();
    ITent createTent();
    ISleepingBag createSleepingBag();
}

```

- This is the heart of the pattern. One factory creates a **whole family** — a tent *and* a sleeping bag.
- `createTent()` **and** `createSleepingBag()` **take NO arguments**. This is deliberate and is the single most important design point (see *Why* below). The factory already knows its family; the caller does not get to influence it per call.
- `getExpeditionTerrain()` lets the provider file each factory under its family.

Concrete factories

```

@Component
@RequiredArgsConstructor
public class DesertGearFactory implements IExpeditionGearFactory {

    private final DesertTent desertTent;
    private final DesertSleepingBag desertSleepingBag;

    @Override public ITent createTent() { return desertTent; }
    @Override public ISleepingBag createSleepingBag() { return desertSleepingBag; }
    @Override public ExpeditionTerrain getExpeditionTerrain() { return ExpeditionTerrain.DESERT; }
}

```

- `@RequiredArgsConstructor` (Lombok) generates a constructor for the two `final` fields, so Spring injects exactly the **desert** products into the **desert** factory.
- `createTent()` returns the desert tent, `createSleepingBag()` returns the desert sleeping bag — a **matched set**. There is no way for this factory to ever return mountain gear. That is the guarantee.

- `MountainGearFactory` is the mirror image with `MountainTent` / `MountainSleepingBag` / `MOUNTAIN`.

ExpeditionGearFactoryProvider — the registry

```
@Component
public class ExpeditionGearFactoryProvider {

    private final EnumMap<ExpeditionTerrain, IExpeditionGearFactory> factoryMap;

    public ExpeditionGearFactoryProvider(List<? extends IExpeditionGearFactory> factories) {
        this.factoryMap = factories.stream().collect(Collectors.toMap(
            IExpeditionGearFactory::getExpeditionTerrain,
            Function.identity(),
            (a, b) -> b,
            () -> new EnumMap<>(ExpeditionTerrain.class)));
    }

    public IExpeditionGearFactory getFactory(final ExpeditionTerrain expeditionTerrain) {
        return factoryMap.get(expeditionTerrain);
    }
}
```

- Structurally identical to `ShapeFactory` in the Factory module, but the values are **factories**, not products. Spring injects every `IExpeditionGearFactory`, and they get filed into an `EnumMap` by the family each reports.
- `getFactory(ExpeditionTerrain)` hands the caller the right concrete factory. From there, everything that factory makes is guaranteed consistent.
- This registry is a convenience layer (a factory-of-factories) so the client can select a family at runtime by enum. It is **not** part of the classic GoF diagram, but it fits this codebase's self-registering `EnumMap` convention.

The demo — AbstractFactoryDesignPattern

```
ExpeditionGearFactoryProvider provider = context.getBean(ExpeditionGearFactoryProvider.class);

IExpeditionGearFactory mountainGearFactory = provider.getFactory(ExpeditionTerrain.MOUNTAIN);
mountainGearFactory.createTent().pitch(); // "Pitching a mountain tent!"
mountainGearFactory.createSleepingBag().unroll(); // "Unrolling a mountain sleeping bag!"

IExpeditionGearFactory desertGearFactory = provider.getFactory(ExpeditionTerrain.DESERT);
desertGearFactory.createTent().pitch(); // "Pitching a desert tent!"
desertGearFactory.createSleepingBag().unroll(); // "Unrolling a desert sleeping bag!"
```

You pick a family **once** (`getFactory(MOUNTAIN)`), and every product after that is mountain gear. That's the payoff.

Why the design decisions

Why do `createTent()` / `createSleepingBag()` take no arguments?

Because **the factory is the choice of family**. The moment you write `createTent(someMaterialOrSize)`, the caller — not the factory — is deciding what comes out, and the family guarantee evaporates. A factory that takes a per-call type argument is just a Factory Method wearing an Abstract Factory costume.

This is exactly the bug a naive implementation would fall into: two concrete factories that are identical and both forward an unrelated quality argument to sub-factories, so the family choice means nothing. The fix is to make each concrete factory own its family outright and drop the arguments.

Why one family enum instead of two (terrain + quality)?

A broken design would have two orthogonal axes — DESERT/MOUNTAIN on the factories and, say, LIGHTWEIGHT/HEAVY-DUTY on the products — that never line up, so a "desert" factory could emit a "heavy-duty" tent and a "lightweight" sleeping bag with no coherence. Collapsing to **one** axis (family) is what makes a factory's output a genuine matched set. If you truly needed terrain *and* load rating, that would be a **2-dimensional** Abstract Factory: four concrete factories (DesertLightweight, DesertHeavyDuty, MountainLightweight, MountainHeavyDuty), each still producing a fully consistent set.

Why inject the concrete products into each factory?

So the matched set is wired at construction time and cannot be violated. `DesertGearFactory` literally only has a `DesertTent` and a `DesertSleepingBag` in its fields — the type system itself forbids it from returning mountain gear.

Why the `EnumMap` provider?

Same reasoning as the Factory module: it lets a caller choose the family at runtime by enum, it auto-discovers new factories (add a new `IExpeditionGearFactory @Component` and it registers itself), and `EnumMap` is the fast, correct container for enum keys. Adding a THIRD family (say `ARCTIC`) means: add `ARCTIC` to the enum, add `ArcticTent / ArcticSleepingBag`, add `ArcticGearFactory` — and **nothing else changes**, including the provider.

Execution flow (as run from `main`)

```
AbstractFactoryDesignPattern.main
├── SpringApplication.run(...)           Spring discovers all @Components
│   ├── DesertTent, DesertSleepingBag, MountainTent, MountainSleepingBag
│   ├── DesertGearFactory ← injected DesertTent + DesertSleepingBag
│   ├── MountainGearFactory ← injected MountainTent + MountainSleepingBag
│   └── ExpeditionGearFactoryProvider ← injected List[Desert..., Mountain...]
│       └── EnumMap { DESERT→DesertGearFactory, MOUNTAIN→MountainGearFactory }
├── provider.getFactory(MOUNTAIN) → MountainGearFactory
│   ├── createTent() → MountainTent.pitch() → "Pitching a mountain tent!"
│   └── createSleepingBag() → MountainSleepingBag.unroll() → "Unrolling a mountain sleeping bag!"
└── provider.getFactory(DESERT) → DesertGearFactory
    ├── createTent() → DesertTent.pitch() → "Pitching a desert tent!"
    └── createSleepingBag() → DesertSleepingBag.unroll() → "Unrolling a desert sleeping bag!"
```

Expected output

```
Pitching a mountain tent!
Unrolling a mountain sleeping bag!
Pitching a desert tent!
Unrolling a desert sleeping bag!
```

Captured from a real run (`java -jar target/Abstract_Factory_Design_Pattern-0.0.1-SNAPSHOT.jar`); the Spring Boot startup/shutdown log lines are omitted above for readability.

How to run

```
# From this module's directory
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean package -DskipTests

# Run the packaged Spring Boot jar (it bundles every dependency, unlike
# plain `target/classes`, which is missing Spring on its classpath)
java -jar target/Abstract_Factory_Design_Pattern-0.0.1-SNAPSHOT.jar

# The app boots an embedded web server (spring-boot-starter-web) and stays
# up after printing the four lines above – stop it with Ctrl-C, or pass
# --server.port=0 to bind an ephemeral port instead of 8080.
```

Drop `-o` (offline) on a first build if the parent reactor's dependencies are not yet in your local Maven repository.

Factory Method vs. Abstract Factory (the one-line rule)

- **Factory Method** makes **one** product, chosen by a key you pass in — `getShape(TRIANGLE)` .
- **Abstract Factory** makes a **family** of products, and the *factory* is the choice — `getFactory(MOUNTAIN).createTent()` / `.createSleepingBag()` .

If every `create...()` call needs a "type" argument, you have Factory Method. If picking the factory already determines everything it produces, you have Abstract Factory.

Note: `SpringApplication.run(...)` boots the container so beans can be discovered and injected; `spring-boot-starter-web` keeps the process alive after the demo prints, so stop it with Ctrl-C. The pattern itself doesn't depend on Spring.